Main Script

DDD 2: Introduction to Programming and Basic Data Analysis

Ramesh Beniwal | Admission ID: BAEC2240006

2026-11-03

Table of contents

Setup and packages 2
Objects, values and names 3
Operations 4
Data frames 5
Vectors and missing values 6
Matrices and arrays 7
Data types, factors and lists 7
Tibbles and data frames 8
Sub-setting 9
Functions 10
Environments 12
For loops 13
Conditions and nested loops 15
For loops with data frames 17
If, else and else if 18
While and repeat loops 19
The map family 20
Real-world data: IHDS 22
Bar plots 24
Contingency table 27
Pie chart 27
Continuous variables 28

The class script from the course. The code is shown as given; it is not run when the site is built, because some lines are exercises or deliberate errors.

Setup and packages

```
#Download and install R and R-Studio (Copy and paste following URL in your
browser)
#(R) https://cran.r-project.org/bin/windows/base/
#(R-Studio) https://posit.co/download/rstudio-desktop/
#we can run a command by Run option or ctrl+enter

#clear the environment
rm(list = ls())
#Set the working directory
#Call our WD path as 'directory'
setwd(directory) #Set new working directory
getwd() #get your working directly (for a cross check)

#(1)for help use ? before command:  ?mean or help(mean)

#(2)Packages: extensions programmed in R by advanced users
# see all packages
library()  #you can manually see this from right bottom panel

#activation of different packages: can be installed in three ways
#a) by command
#b) from Tools option at top
#c) from Packages option on bottom right panel

#Install packages
install.packages("lobstr") #Check data structure
install.packages("rio") #import, export almost any file format

#Load packages
library(lobstr,rio)

?lobstr #check description of this package

#install different packages
#install.packages(c("package1", "package1"))

#see packages currently loaded
search()
```

Clears the workspace, sets the working directory and installs and loads packages.

Objects, values and names

```
#(3)Objects, values and names
# <- is called assign operator. Left is object name and right is value
# data flows flows from right → left

## create an object (e.g., any vector) and assign it any name
x<-c(1,2,3,4)
y<-x  #this does not create new object (i.e., vector (1,2,3,4)), only get another
binding y

#check the adress of object
lobstr::obj_addr(x)
lobstr::obj_addr(y)

#not a valid name
_a <- 5

#can use "" or ''
"_a"<-5
'_b'<-5

#Question: comment on a,b,c and d (are these different objects)
a <- 1:10
b <- a
c <- b
d <- 1:10
#check
list_1 <- list(a, b, c, d)
obj_addrs(list_1) #without calling the package "lobstr"


##copy-on-modify (modify some value in y)
y[[3]] <- 5 #change the third one
y[[5]]<- 6 #add new one
#now these are two different objects
obj_addr(x)
obj_addr(y)

#list and copy-on-modify with list command
list_2<-list(1,2,3,4)
list_3<-list_2
obj_addr(list_2)
obj_addr(list_3)
```

```r
#modify list_3
list_3[[4]]<-5
list_3[[5]]<-6
obj_addr(list_2)
obj_addr(list_3)
```

Two names can point to one object; R copies it only when one of them is modified (copy-on-modify).

Operations

```r
##Operations in R
# Define two numbers (number vs number)
a <- 10
b <- 3
# Addition
a + b
# Subtraction
a - b
# Multiplication
a * b
# Division
a / b
# Greater than
a > b
# Less than
a < b

# Define two vectors (vector vs vector)
x <- c(2, 4, 6, 8)
y <- c(1, 2, 3, 4)
# Addition (element-wise)
x + y
# Subtraction (element-wise)
x - y
# Multiplication (element-wise)
x * y
# Division (element-wise)
x / y
# Greater than (element-wise comparison)
x > y
# Less than (element-wise comparison)
```

```
x < y

#vector and number
x+5 #same for -,*,/
x>5 #same for logical operator

#Columns of unequal length
x1<-c(1,2,3,4)
x2<-c(7,8)
x3<-c(7,8,1)

x1+x2
x1+x3
```

Arithmetic and comparison on numbers and vectors; vectors of unequal length are recycled.

Data frames

```
##Data frames
#way_1
data_1 <- data.frame(id = c(1,2,3), age = c(20,19,23), gender=c("m","f","m"))
#way_2
id= c(1,2,3)
age= c(20,19,23)
gender=c("m","f","m") #characters are in ""
data_2<-data.frame(id,age,gender)

#data_3 same as data_1
data_3<-data_1

#list of data
list_d<-list(data_1,data_2,data_3)
#check address
obj_addrs(list_d)

#change one column or row by [row number,column number]
data_3[, 2] <- data_3[, 2] * 2
#can no not be done with character
data_3[, 3] <- dacta_3[, 3] * 2

#change only one entry of variable "age"
data_1[1, "age"] <- 29
```

```
install.packages("rio")
library(rio)
#export data-frame in excel format
rio::export(data_1, "data/my_data_1.csv")

##Exercise
#generate a data frame called students_df for a group of 10 students: id, age,
gender, state
#suppose age of the second student is recorded incorrectly. Write an R command
to correct it.
#export both datasets in excel format for a cross check
```

Two ways to create a data frame, changing a column or a single entry, and exporting.

Vectors and missing values

```
###################
##Vectors
lg_v <- c(TRUE, FALSE) #logical
in_v <- c(1L, 6L, 10L) #integer
db_v <- c(1, 2.5, 4.5) #double
ch_v <- c("these are", "some strings") #character
#check type
typeof(lg_v)

v1<-c(1,2,3,4)
v2<-c(1L,2L,3L,4L)
#logical vector more
as.numeric(lg_v)
sum(lg_v)
mean(lg_v)
#make chnages in logical vector
lg_v[[3]]<-FALSE
sum(ch_v)

##Missing values  --- NA
m <- c(NA, 5, NA, 10)
#check which value is missing
is.na(m)
is.na(data_1)
```

The four vector types and `NA`.

Matrices and arrays

```
##Matrices and Arrays
mr<-matrix(1:6, c(2, 3)) #c(rows,cols)

ar<-array(1:12, c(2, 3, 2)) #(row,cols,layers)

dim(mr) #check the dimensions

#dim can be used either for matrix or array
z <- 1:6
dim(z) <- c(3, 2,1) #we can layer for array
z

#matrix can be of 1Xn dimensions (will look like a column)
c1<-c(1,2,3,4)
m1<-matrix(1:4,c(4,1))
str(c1) #check the structure
str(m1)

#comment on the following arrays
ar1 <- array(1:5, c(1, 1, 5))
ar2 <- array(1:5, c(1, 5, 1))
ar3 <- array(1:5, c(5, 1, 1))

c4<-c(1:5) #how is c4 different from ar2
```

`dim()` sets rows, columns and layers.

Data types, factors and lists

```
##Data Types
#(1) Numerical: Discrete and Continuous
#(2) Categorical: Nominal and Ordinal

#Factors (for categorical data)
v1<-c( "M", "P", "S", "H", "P", "S", "M", "M") #levels are automatically decide
by alphabetic order
f1<-factor(v1) #or f<-factor(c("a", "b"))
typeof(f1) #check what type it is

#define levels from here
f2<-factor(v1, levels = c("P", "M", "S", "H"), ordered=T)
```

```
as.character()
as.numeric()  #can be used to check distinction between character vector and
a factor

##Lists
l3 <- list(
  1:3,
  a=c("a","b"),
  c(TRUE, FALSE, TRUE),
  c(2.3, 5.9)
)
typeof(l3)
str(l3)
```

Numerical and categorical data; factors with levels; lists.

Tibbles and data frames

```
#data frame and tibble
install.packages("tibble")
library(tibble)

df1 <- data.frame(x = 1:3, y = letters[1:3])
df2 <- tibble(x = 1:3, y = letters[1:3])
df1
df2

tibble(
  x = 1:3,
  y = x * 2
)     #refer to variables created during construction

data.frame(
  x = 1:3,
  y = x * 2
)   #doesn't refer to variables created during construction


#row names
df3 <- data.frame(
  age = c(35, 27, 18),
  gender = c("M", "F", "F"),
  row.names = c("aa", "bb", "cc")
```

```
)

#we can get the details of only specific row now
df3["bb",1] #first column
df3["bb",]  #all columnns

rownames(df3)
colnames(df3) #or names(df3)

nrow(df3)
ncol(df3) #or length(df3)

#add new col to data
df2$z<-3:5

library(dplyr)  #for data manipulation

#rename variables of data frame  %>% or |> is the pipe operator
df2 <- df2 %>%
  rename(
    xn = x,
    yn = y
  )  #new name = old name
```

Row and column names, adding a column and renaming with `dplyr`.

Sub-setting

```
##Sub-setting
#in vectors
rm(x)
x <- c(2.1, 4.2, 3.3, 5.4)

y<-x[c(1,2)] #get only 1st and 2nd obs
x[-c(1,2)] #get all except 1st and 2nd obs
x[c(1.3, 1.9)] #real numbers get the integers places
x[x > 3]
x[2] #will keep only second element
x[] #empty returns all

#in data-frames
df <- data.frame(x = 1:3, y = 4:6, z = letters[1:3])
```

```r
library(dplyr)
#select rows
df1<- df[df$x == 2, ] #selects only row 2
df[c(1,3), ] #selects row 1 and 3

df3<-df[c("x", "z")] #all rows, only x and y

df[c(1,3), c("x", "z")] #row 1 and 3, only x and y

##Exercise
#Create a data frame students with: id = 1:4, age = c(20, 22, 21, 23), gender =
c("F", "M", "F", "M")
#Rename:age to age_years, gender to gender_binary
#Add a new column passed with values: TRUE, TRUE, FALSE, TRUE
#Select: Only students whose age_years is greater than 21, only columns id and sex
```

Selecting elements of a vector and rows and columns of a data frame.

Functions

```r
##Functions##
df <- tibble(
  a = rnorm(5),
  b = rnorm(5),
  c = rnorm(5),
  d = rnorm(5),
)


#make changes in the data
df<-df |> mutate(
  a = (a - mean(a, na.rm = TRUE)),
  b = (b - mean(b, na.rm = TRUE)),
  c = (c - mean(c, na.rm = TRUE)),
  e = (d - min(d, na.rm = TRUE))
)


#Function (basics)
add_1 <- function(x) {
  x + 1
}  #three parts are there, name (add_1), argument(x), and body{}
```

```r
add_1(6)

sq_x <- function(x) {
  x^2
}

sq_x(7)


x_p <- function(x,p=3) {
  x^p
}

x_p(2)

add_xy <- function(x,y) {
  x + y
}

add_xy(x=c(1,2,4,5),y=c(4,6,8,9))


#nowf2<- lets go back to our example
mean_dev <- function(x) {
  (x - mean(x, na.rm = TRUE))
}

#do it for all a,b,c,d
df |>
  mutate(
    across(c(a, b, c, d), mean_dev),
    e=d - mean(d, na.rm = TRUE),
    f=sum(a),
    g=mean(b)
  )

#Exercise for the above data frame:
#define an function for mean of variable and do the above exercise
#define an function for x-min/min-max of variable and do the above exercise

f2#Let's do so some standarization with function()
snv<-function(x){(x-mean(x))/sd(x)} #run it for any variable
```

```
#what if you will not use na.rm=T and your data has missing values
#(will functions run correctly?)
```

A function has a name, arguments and a body; used here to standardise variables.

Environments

```
##Environments##
install.packages("rlang") #for env rlang package is required
library(rlang)


#Check current environment
environment()

ls()

rm(list=ls())

#clear the current env
rm(list = ls())

#Creating objects in Global Environment
x <- 10
y <- 20
name <- "Rohit"
#check what is in current env
ls()

#crate new env
e1 <- new.env()
#adding objs to new env
e1$a <- 100
e1$b <- 200
#check the list in this new env
ls(e1)
e1$a
#check parent env
parent.env(e1)

#from rlang package
e2 <- env(
  a = 120,
```

```
  b = 250,
)

e2$a
e2$b

ls(e2)
rm(e1$a)


#function creates it is own env
f <- function() {x <- 15
y <- 10
x + y
}

f()

#lexical scoping (if object is not found inside function, will go to global env)
a <- 100

g <- function() {
  a + 1
}

g()
```

Global and new environments, and lexical scoping.

For loops

```
###Loops###

#A loop repeats the same task multiple times
#same task again and again
x1 <- 1^2
x2 <- 2^2
x3 <- 3^2

#Main loops in R: (1)for (2)while (3)repeat loops
#For loop :
#for(values in sequence){
code that will repeat for all vlaues
```

```r
}

#for vector
for(i in 1:3){
  print(i^2)
}   #print will only show results

for(i in c(1,2,3)){
  print(i^2)
}

#to use the results we need to save results from for loop
for(i in c(1,2,3)){
  x<-(i^2)
}  #this will only consider last value i.e., 3

#gen empty vector of length 3 and then x[i] will gen i^2
x<-numeric(3)

for (i in 1:3) {
  x[i] <- i^2
}

#for existing vector
x<-c(3,2,5)
for(i in x){print(i^2)
}

#this will overwrite x
for(i in x){x[i]<-(i^2)
}

#to store the loop results for x in y
y<-numeric(3)

for (i in x) {y[i]<-i^2}

#how to solve the issue
x <- c(3, 2, 5)
y <- numeric(length(x))

for (i in 1:length(x)) {
  y[i] <- (x[i])^2
```

```
}

for (i in 1:length(x)) {
  y[i] <- f(i)
}

y

#we can have more than one repeating codes
for(i in x){print(c(i,i^2,i^3))
}

for(i in x){z<-i^2
print(c(i,z))}

#write for loop and print z as x-mean(x)/sd(x)
```

Repeating a task, and saving the result in a vector.

Conditions and nested loops

```
#ifelse for loop
x<-c(-1,2,3)
#if else can not work for x
if(x<0){print("negative")} else
{print("positive")}

#for loop can be used(Conditional logic inside loops)
for(i in x){if(i<0){print("negative")} else
{print("positive")}
}


#Nested for loop
for(i in vec 1) {
  for(j in vect 2){
    for(k in vect n) {
      code here
    }#n
  }#2
} #1

#for two vectors sum of cross elements
```

```r
for(i in 1:3){
  for(j in 1:2){
    print(i+j)
  }
}

#write the one for three vectors

#nested loop for matrices (5x4)
mat <- matrix(0, nrow = 5, ncol = 4)
dim(mat)


for (i in 1:5) {
  for (j in 1:4) {
    mat[i, j] <- i * j
  }
}


#loops can be avoided
x<-c(1,2,3)
x<-x^2

#loop with function

f <- function(x) {
  x^2
}

for (i in 2:4) {
  print(f(i))
}

#save these results
x <- numeric(3)
for (i in 2:4){
  x[i]<-f(i)
}
x


x <- numeric(3)
```

```r
for (i in c(2,3,5)){
  x[i]<-f(i)
}

x <- c(3, 2, 5)
y <- numeric(length(x))

for (i in 1:length(x)) {
  y[i] <- f(x[i])
}




#how can we avoid this loop here, write direct code

#exercises
#(1) Gen a vector of length 5, multiply each element by 3 and save the result
in new vector
#(2) use nested loop and gen output like
[1] 1 1
[1] 1 2
[1] 1 3
[1] 2 1
[1] 2 2
[1] 2 3
[1] 3 1
[1] 3 2
[1] 3 3
```

`if` inside a loop, and loops inside loops, including a matrix.

For loops with data frames

```r
#for loop in data frame
df <- data.frame(
  id = 1:5,
  income = c(1000, 1500, 1200, 1800, 1600)
)

x <- numeric(5)
#change x as per any column
for (i in 1:nrow(df)) {
```

```
  x[i] <- df$income[i]+500
}

x

#add this newly generated x to df
#how can you add x directly without writing for loop
```

Adds a number to a column with a loop.

If, else and else if

```
#if, else, ifelse , elseif conditions
#logical operators
1>2
2<3
2==3
2>=1
5<=4
5!=2

#if

#if(condition){run if
True} #code will run only if condition is true

x<-10

if(x>10){
  print("x is 10")
}

#if else
if (condition 1) { code if true
} else {
  code if false
}


#pass or fail
marks <- 42

if (marks >= 40) {
```

```
  print("Pass")} else {
    print("Fail")
  }

#even/odd number
x <- 7

if (x %% 2 == 0) {
  print("Even number")
} else {
  print("Odd number")
}

#gen a vector of even and odd numbers and run if else condition for that vector

marks <- 75

if (marks >= 90) {
  print("Grade A")
} else if (marks >= 75) {
  print("Grade B")
} else if (marks >= 60) {
  print("Grade C")
} else {
  print("Fail")
}

#run this for a vector
```

Pass or fail, even or odd, and grades.

While and repeat loops

```
#while loop
while(logical condition) {command here}

x<-1
while(x<5){print(x)
  x<-x+1}
x<-1
while(x<5){x<-x+1
print(x)
}  #position of code matters here
```

```
#store the results
y<-c() #gen null vector

#overwrite this null y as per the while loop conditions
x<-1
while(x<5){y<-c(y,x)
x<-x+1
}


#exercise
x <- 2
while (x <= 10) {
  print(x)
  x <- x + 2
}
#(i)Predict the output without running the code
#(ii)What is the starting value of x?
#(iii)When does the loop stop?
#(iv)Modify the code so that it prints odd numbers less than 14

#repeat loop
x<-10
repeat{x=x+2
print(x)
}

#put a break
x<-10
repeat{x=x+2
print(x)
if(x==20) break
}
```

`while` runs while a condition holds; `repeat` runs until `break`.

The map family

```
#The map function (runs function for data)
#basic syntax
map(x,f) #x is data and f is function
```

```r
map() makes a list.
map_lgl() makes a logical vector
map_int() makes an integer vector
map_dbl() makes a double vector
map_chr() makes a character vector
library(purrr)

f1 <- function(x){x*3}
#first one
map(1:3, f1)
m<-map_dbl(1:3, f1) #or function inside map()
map_dbl(1:3, function(x){x*3})
map_dbl(1:3, ~.x*3)
#long code
f1(1)
f1(2)
f1(3) # or we will write for loop

#for many vectors (pmap)
pmap_dbl(list(c(1,2,5),c(2,5,7)), function(x,y){x+y})
pmap_dbl(list(c(1,2,5),c(2,5,7)), function(x,y){mean(c(x,y))})


#for dataframe
df<-tibble::tibble(a=rnorm(5), b=rnorm(5),c=rnorm(5))
#what if we need to find median of vaaibles of df
#way 1
mean(df$a)
mean(df$b)
mean(df$3)
#way 2
for (i in seq_along(df)) {
  print(mean(df[[i]]))
}
#way 3 (map)
map_dbl(df, mean) #or
df|>map_dbl(mean)
#for only selected columns
df|>select(a,b)|>map_dbl(mean)


#lappy(similar to map, but gives result only as list)
#sapply(gives output as vector)
```

```r
l1<-list(a=c(1,2,4),b=c(3,5,6))
lapply(l1,mean)
sapply(df,mean)
df|>select(a,b)|>lapply(mean)
```

`map()`, `pmap()`, `lapply()` and `sapply()`.

Real-world data: IHDS

```r
df1<-import("C:\\Users\\BilalAhmadBhat\\OneDrive - Azim Premji Foundation\\
            Teaching\\4_Inro_Prog_Data_25_26_II\\
            R_work\\data\\raw_data\\ihds\\DS0002\\36151-0002-Data.dta")



###Real World Data###

library(rio)
library(dplyr)
library(labelled)
library(ggplot2)
library(stargazer)

#import ihds hhd data
df1<-import("data\\raw_data\\ihds\\DS0002\\36151-0002-Data.dta")

##select required vars and export this data
df1|>select(ID11,ID13,FM1,ID18C)|>export("data\\auxiliary_data\\hhd_1.dta")
#import this small file
df2<-import("data\\auxiliary_data\\hhd_1.dta")

## With out questionnaire check var details
install.packages("labelled")
library(labelled)
var_label(df2$ID11)
#change existing var labels
var_label(df2$ID11) <- "religion of houesehold head"

##rename req vars
df2<-df2|>rename(land=FM1,religion=ID11,education=ID18C)

##clean the data for vars
unique(education)
```

```r
table(education)
attributes(education)
df2|>count(education) #preferable way

#let 16 years of educ is outlier (replace by NA)
df2 <- df2 |>mutate(education=na_if(education,16))
#or
df2<-df2|>mutate(education=ifelse(education==16,NA,education))


#create dummy from continuous variables
df2<-df2|>mutate(any_educ=ifelse(education>0,1,0|is.na(education)))
#add label for this now
var_label(df2$any_educ)<-"if any years of education"

#categories of education (1,2,3)
df2<- df2 |>
  mutate(educ_cat= case_when(
    education <=5 ~ 1,
    education >5  & education <= 12 ~ 2,
    education >12 ~ 3
  ))

#add labels to these (assign some name to these categories)
val_labels(df2$educ_cat) <- c(
  Primary = 1,
  High = 2,
  Higher = 3
)

#keep for some values (filter for outliers)
df2<-df2|>filter(education<=15&education>0|is.na(education))

#save the cleaned data for future use
df2|>export("data\\auxiliary_data\\hhd_2.dta")

#now begin analysis on hhd_2 data
df2<-import("data\\auxiliary_data\\hhd_2.dta")



######Data Analysis######
##Summary Statistics
```

```
library(stargazer)
df2|>select(education, land)|>stargazer(
  type = "text", title = "Summary Stat 1", # or "html", "latex"
  summary.stat = c("n", "mean", "sd", "min", "max"),
  out = "results/summary_stats_1.rtf")
```

Imports the IHDS data, keeps four variables, labels and cleans them, and prepares the summary table.

Bar plots

```
###Data visualization
library(ggplot2)
#basic syntax

ggplot(data,aes(x=var1,y=var2))+
  geom_yourchoice()

##bar chat [total count to begin with]
ggplot(df2, aes(x = educ_cat)) +
  geom_bar()  #will not include NAs and will consider x as continuous

ggplot(df2, aes(x = factor(educ_cat))) +
  geom_bar()  #will include NAs and will consider x as categorical

ggplot(df2, aes(x = factor(educ_cat), fill=factor(educ_cat))) +
  geom_bar()  #add legend and colors

ggplot(df2|>filter(!is.na(educ_cat)),       aes(x       =       factor(educ_cat,
levels=c(1,2,3),labels=c("Low","high","higher")), fill=factor(educ_cat))) +
  geom_bar()  #get rid of NAs

#this will remove legend
ggplot(df2|>filter(!is.na(educ_cat)),
aes(x = factor(educ_cat,levels=c(1,2,3),labels=c("Low","High","Higher")), fill
= factor(educ_cat))) +
  geom_bar()

ggplot(df2|>filter(!is.na(educ_cat)),
aes(x = factor(educ_cat,levels=c(1,2,3),labels=c("Low","High","Higher")), fill
= factor(educ_cat))) +
  geom_bar() + theme(legend.position = "none")
```

```r
#edit axis labels and titles: labs()
ggplot(df2|>filter(!is.na(educ_cat)),
aes(x = factor(educ_cat,levels=c(1,2,3),labels=c("Low","High","Higher")), fill
= factor(educ_cat))) +
  geom_bar() + labs(
    title = "Figure 1: Bar Plot",
    x = "Education Category",
    y = "Count",
    fill = "Educ cat"
  ) + theme(legend.position = "none") + theme_minimal() + theme()

#over all theme and position of title and legend
ggplot(df2|>filter(!is.na(educ_cat)),
aes(x = factor(educ_cat,levels=c(1,2,3),labels=c("Low","High","Higher")), fill
= factor(educ_cat))) +
  geom_bar() + labs(
    title = "Figure 1: Bar Plot",
    x = "Education Category",
    y = "Count",
    fill = "Educ cat"
  ) +  theme_minimal() + theme(plot.title = element_text(hjust = 0.5),
                               legend.position = "")


#Now let's do proportions
df2|>filter(!is.na(educ_cat))|>
  count(educ_cat)|>mutate(prop=n/sum(n))|>
  ggplot(aes(x = factor(educ_cat,levels=c(1,2,3),
                              labels=c("Low","High","Higher")), y=prop, fill =
factor(educ_cat))) +
  geom_col() + labs(
    title = "Figure 1: Bar Plot",
    x = "Education Category",
    y = "Count",
    fill = "Educ cat"
  ) +  theme_minimal() + theme(plot.title = element_text(hjust = 0.5),
                               legend.position = "")


install.packages("scales")
library(scales)

#convert y label into %
```

```r
df2|>filter(!is.na(educ_cat))|>
  count(educ_cat)|>mutate(prop=n/sum(n))|>
  ggplot(aes(x = factor(educ_cat,levels=c(1,2,3),
                               labels=c("Low","High","Higher")), y=prop, fill =
factor(educ_cat))) +
  geom_col() + scale_y_continuous(labels = percent) + labs(
    title = "Figure 1: Bar Plot",
    x = "Education Category",
    y = "Percent",
    fill = "Educ cat"
  ) +  theme_minimal() + theme(plot.title = element_text(hjust = 0.5),
                               legend.position = "")

#add percent at top of bar (by geom_text)
df2|>filter(!is.na(educ_cat))|>
 count(educ_cat)|>mutate(prop=n/sum(n),top_lab=paste0(round(prop*100,1),"%"))|
>
  ggplot(aes(x = factor(educ_cat,levels=c(1,2,3),
                               labels=c("Low","High","Higher")), y=prop, fill =
factor(educ_cat))) +
  geom_col() + scale_y_continuous(labels = percent) +
  geom_text(
    aes(label = top_lab),
    vjust = -0.05,   # moves text above bars
    size = 2
  ) +
  labs(
    title = "Figure 1: Bar Plot",
    x = "Education Category",
    y = "Percent",
    fill = "Educ cat"
  ) +  theme_minimal() + theme(plot.title = element_text(hjust = 0.5),
                               legend.position = "")


#Side by side
df2|>filter(!is.na(educ_cat),!is.na(land))|>
  count(educ_cat,land) |> mutate(prop=n/sum(n))|>
  ggplot(aes(x = factor(educ_cat,levels=c(1,2,3),
                                labels=c("Low","High","Higher")), y=prop, fill
= factor(land,levels=c(0,1),
                                                        labels=c("No","Yes")))) +
  geom_col(position = "dodge") + scale_y_continuous(labels = percent) + labs(
```

```
    title = "Figure 1: Bar Plot",
    x = "Education Category",
    y = "Percent",
    fill = "Land"
  ) +  theme_minimal() + theme(plot.title = element_text(hjust = 0.5),
                               legend.position = "right")
```

Counts, proportions, percentages and side-by-side bars.

Contingency table

```
##Contingency Tables
library(janitor)
library(flextable)
ct_1<-df2|> mutate(across(c(educ_cat,land), as.factor))|>
  tabyl(educ_cat,land) |>
  adorn_percentages("all") |>
  adorn_pct_formatting() |>
  adorn_ns()

ct_2<-flextable(ct_1)
save_as_docx(ct_2, path="results/contg_1.docx")
```

Cross table saved to Word.

Pie chart

```
#prepare a pie chart for "educ_cat" (title, legend and percentages(outside the
slices) should be there)
#simple pie chart
df2|>
  filter(!is.na(educ_cat))|>
  count(educ_cat)|>
  mutate(
    prop = n / sum(n)
  )|>
  ggplot(aes(x = "", y = prop, fill = factor(educ_cat))) +
  geom_col() + coord_polar(theta = "y") +  theme_void()

#final pie chart
df2|>
  filter(!is.na(educ_cat))|>
  count(educ_cat)|>
```

```
  mutate(
    prop = n / sum(n),
    values = paste0(round(prop * 100, 1),"%")
  )|>
  ggplot(aes(x = "", y = prop, fill = factor(educ_cat,levels=c(1,2,3),
                                             labels=c("Low","High","Higher")))) +
  geom_col() + coord_polar(theta = "y") +
  geom_text(
    aes(label = values),
    position = position_stack(vjust = 0.5),
    size = 2
  ) + theme_void() + labs(fill = "Educ Cat")
```

Simple and final version with labels.

Continuous variables

```
###now the continuous variables
#Box plots
ggplot(df2, aes(y=education)) + geom_boxplot(fill="orange") +
  labs(title="Box Plot 1", y="Years of education")

#for COPC
df2|>filter(!is.na(COPC))|>ggplot(aes(y=COPC)) + geom_boxplot(fill="orange") +
  labs(title="Box Plot 1", y="Consumption")
#remove out-liers and check
df2|>filter(!is.na(COPC))|>filter(COPC<30000)|>
  ggplot(aes(y=COPC)) + geom_boxplot(fill="orange") +
  labs(title="Box Plot 1", y="Consumption")

#count of how many observations are geater than some value of x
sum(df2$COPC > 30000, na.rm = TRUE) #how many obs of COPC are >50000

#histogram
ggplot(df2, aes(education, fill="red")) + geom_histogram(bins=8) +
  labs(title="hist_1", x="Education Years", y="Count") +
  scale_x_continuous(breaks=c(0,4,8,12,16)) + labs(fill = "Educ") +
  theme(legend.position = "")
```

Box plot, outlier check and histogram.